

Contact and Friction Derivations for GeoTaichi

2026-04-26

1. Scope

This note summarizes the formulas currently used in GeoTaichi for:

1. IPC-style friction in `src/igampm/MPM/ipc_mpm.py`, with the reference implementation taken from `phyverse`.
2. Particle–NURBS-surface contact and friction in `src/igampm/Coupling/coupling.py`.

The purpose is implementation correspondence. The derivations below therefore follow the actual code paths in:

- `src/igampm/IPC.py`
- `src/igampm/MPM/ipc_mpm.py`
- `src/igampm/Coupling/coupling.py`

2. Scalar Barrier and Friction Building Blocks

2.1 Barrier

In `src/igampm/IPC.py`, the scalar barrier energy is

$$B(d) = \begin{cases} -\kappa(d - \hat{d})^2 \log\left(\frac{d}{\hat{d}}\right), & d < \hat{d}, \\ 0, & d \geq \hat{d}. \end{cases}$$

Its first and second derivatives are

$$\begin{aligned} B'(d) &= \kappa \left[-2(d - \hat{d}) \log\left(\frac{d}{\hat{d}}\right) - \frac{(d - \hat{d})^2}{d} \right], \\ B''(d) &= \kappa \left[-2 \log\left(\frac{d}{\hat{d}}\right) - 4 + \frac{4\hat{d}}{d} + \frac{(d - \hat{d})^2}{d^2} \right]. \end{aligned}$$

If the contact distance is represented through a squared-distance map

$$s(\mathbf{q}) = d(\mathbf{q})^2,$$

then the code uses

$$\nabla d = \frac{1}{2d} \nabla s,$$

and therefore

$$\nabla \Psi_b = B'(d) \nabla d = \frac{B'(d)}{2d} \nabla s.$$

The Hessian implemented in `Barrier.hessian()` is

$$\nabla^2 \Psi_b = \frac{1}{4} \left(\frac{B''(d)}{d^2} - \frac{B'(d)}{d^3} \right) \nabla s \nabla s^T + \frac{1}{2} \frac{B'(d)}{d} \nabla^2 s.$$

2.2 Friction

The smoothed scalar friction function in `src/igampm/IPC.py` is

$$f_0(\bar{v}, \varepsilon_v, \Delta t) = \begin{cases} \bar{v}\Delta t, & \bar{v} \geq \varepsilon_v, \\ \frac{(\bar{v}\Delta t)^2 \left(-\frac{\bar{v}\Delta t}{3} + \varepsilon_v \Delta t \right)}{(\varepsilon_v \Delta t)^2} + \frac{\varepsilon_v \Delta t}{3}, & \bar{v} < \varepsilon_v. \end{cases}$$

The auxiliary terms used in code are

$$f_1(\bar{v}) = \begin{cases} \frac{1}{\bar{v}}, & \bar{v} \geq \varepsilon_v, \\ \frac{-\bar{v} + 2\varepsilon_v}{\varepsilon_v^2}, & \bar{v} < \varepsilon_v, \end{cases}$$

$$f_2(\bar{v}) = \begin{cases} -\frac{1}{\bar{v}^2}, & \bar{v} \geq \varepsilon_v, \\ -\frac{1}{\varepsilon_v^2}, & \bar{v} < \varepsilon_v. \end{cases}$$

For a frozen contact normal $\mathbf{n}$, define the tangential projector

$$\mathbf{T} = \mathbf{I} - \mathbf{n}\mathbf{n}^T.$$

Given a relative displacement increment $\Delta \mathbf{x}$, the code forms

$$\bar{\mathbf{x}} = \mathbf{T}^T \Delta \mathbf{x}, \quad \bar{d} = \|\bar{\mathbf{x}}\|.$$

The friction energy is

$$\Psi_f = \mu\lambda f_0(\bar{d}, \varepsilon_v, \Delta t).$$

The *implemented* gradient helper in `Friction.gradient()` is

$$\mathbf{g}_{\Delta x}^{\text{code}} = -\mu\lambda f_1(\bar{d}) \frac{\mathbf{T}\bar{\mathbf{x}}}{\bar{d}}, \quad (\bar{d} \neq 0),$$

and the *implemented* Hessian helper in `Friction.hessian()` is

$$\mathbf{H}_{\Delta x}^{\text{code}} = \mathbf{T} \left[\mu\lambda \left(\frac{f_1(\bar{d})}{\bar{d}} \mathbf{I} + \frac{f_2(\bar{d})\bar{d} - f_1(\bar{d})}{\bar{d}^3} \bar{\mathbf{x}}\bar{\mathbf{x}}^T \right) \right] \mathbf{T}^T, \quad (\bar{d} \neq 0).$$

The code then contracts these local expressions with the corresponding kinematic Jacobians.

3. IPC-MPM Friction Aligned with phyverse

3.1 Frozen normal and tangential relative motion

The `phyverse` implementation computes a friction weight

$$\lambda = -B'(d), \quad \mu\lambda = \mu\lambda,$$

and freezes the contact normal during one friction linearization. GeoTaichi follows the same idea for:

- MPM point–half-space contact.
- MPM point–point contact.

3.2 MPM point–half-space friction

Let $\mathbf{x}_p$ be the current MPM surface point, $\hat{\mathbf{x}}_p$ the frozen reference point, $\mathbf{n}$ the half-space normal, and $\mathbf{v}_{hs}$ the half-space velocity. The code uses

$$\mathbf{T} = \mathbf{I} - \mathbf{n}\mathbf{n}^T, \quad \Delta\mathbf{x}_{hs} = \mathbf{x}_p - \hat{\mathbf{x}}_p - \Delta t \mathbf{v}_{hs},$$

$$\bar{\mathbf{v}}_{hs} = \mathbf{T}^T \frac{\Delta\mathbf{x}_{hs}}{\Delta t}.$$

Then

$$\Psi_{hs} = \mu\lambda f_0(\|\bar{\mathbf{v}}_{hs}\|, \varepsilon_v, \Delta t).$$

If the point is interpolated from grid unknowns by

$$\mathbf{x}_p = \sum_a N_a \mathbf{x}_a,$$

then the code assembles

$$\frac{\partial \Psi_{hs}}{\partial \mathbf{x}_a} = N_a \mathbf{g}_{hs}, \quad \frac{\partial^2 \Psi_{hs}}{\partial \mathbf{x}_a \partial \mathbf{x}_b} = N_a N_b \mathbf{H}_{hs},$$

where $\mathbf{g}_{hs}$ and $\mathbf{H}_{hs}$ are the local expressions evaluated from the helper functions above.

3.3 MPM point–point friction

For two frozen contact points i, j , let

$$\mathbf{n}_{ij} = \frac{\mathbf{x}_i - \mathbf{x}_j}{\|\mathbf{x}_i - \mathbf{x}_j\|}, \quad \mathbf{T}_{ij} = \mathbf{I} - \mathbf{n}_{ij}\mathbf{n}_{ij}^T,$$

$$\Delta\mathbf{x}_{ij} = (\mathbf{x}_i - \hat{\mathbf{x}}_i) - (\mathbf{x}_j - \hat{\mathbf{x}}_j), \quad \bar{\mathbf{v}}_{ij} = \mathbf{T}_{ij}^T \frac{\Delta\mathbf{x}_{ij}}{\Delta t}.$$

The energy is

$$\Psi_{ij} = \mu\lambda f_0(\|\bar{\mathbf{v}}_{ij}\|, \varepsilon_v, \Delta t).$$

If

$$\mathbf{x}_i = \sum_a N_a^{(i)} \mathbf{x}_a, \quad \mathbf{x}_j = \sum_b N_b^{(j)} \mathbf{x}_b,$$

then the assembled gradient blocks are

$$\frac{\partial \Psi_{ij}}{\partial \mathbf{x}_a^{(i)}} = N_a^{(i)} \mathbf{g}_{ij}, \quad \frac{\partial \Psi_{ij}}{\partial \mathbf{x}_b^{(j)}} = -N_b^{(j)} \mathbf{g}_{ij}.$$

The Hessian blocks are

$$\begin{aligned} \mathbf{H}_{ii}^{ab} &= N_a^{(i)} N_b^{(i)} \mathbf{H}_{ij}, & \mathbf{H}_{jj}^{ab} &= N_a^{(j)} N_b^{(j)} \mathbf{H}_{ij}, \\ \mathbf{H}_{ij}^{ab} &= -N_a^{(i)} N_b^{(j)} \mathbf{H}_{ij}, & \mathbf{H}_{ji}^{ab} &= -N_a^{(j)} N_b^{(i)} \mathbf{H}_{ij}. \end{aligned}$$

This is exactly the sign structure implemented in the corrected GeoTaichi code.

4. MPM-IGA Coupling in `coupling.py`

4.1 Kinematics

The coupling module considers a particle point against an IGA NURBS surface. Let

$$\mathbf{c}(u, v) = \sum_a N_a(u, v) \mathbf{P}_a$$

be the surface point associated with the closest-point parameters (u, v) , and let $\mathbf{x}_p$ be the particle center with radius R .

Define the pointer vector

$$\mathbf{r} = \mathbf{c}(u, v) - \mathbf{x}_p,$$

and the gap

$$d = \|\mathbf{r}\| - R.$$

The code stores the closest-point geometry through:

- NURBS basis values N_a .
- First derivatives of the basis with respect to u, v .
- Surface tangents and curvatures.
- The closest-point normal $\mathbf{n} = \mathbf{r}/\|\mathbf{r}\|$ for friction initialization.

4.2 Barrier energy for MPM-IGA contact

The barrier energy used in `calculate_barrier_energy()` is

$$\Psi_b = B(d).$$

The implementation differentiates through the squared-distance map

$$s = \|\mathbf{r}\|^2.$$

4.2.1 Implicit differentiation of the closest-point map

The missing intermediate step is that (u, v) are not independent variables. They are defined implicitly by the closest-point orthogonality conditions

$$g_1(u, v, \mathbf{x}_p, \{\mathbf{P}_a\}) = \mathbf{r} \cdot \mathbf{c}_{,u} = 0, \quad g_2(u, v, \mathbf{x}_p, \{\mathbf{P}_a\}) = \mathbf{r} \cdot \mathbf{c}_{,v} = 0.$$

Here

$$\mathbf{c}_{,u} = \frac{\partial \mathbf{c}}{\partial u}, \quad \mathbf{c}_{,v} = \frac{\partial \mathbf{c}}{\partial v},$$

and

$$\mathbf{c}_{,uu} = \frac{\partial^2 \mathbf{c}}{\partial u^2}, \quad \mathbf{c}_{,vv} = \frac{\partial^2 \mathbf{c}}{\partial v^2}, \quad \mathbf{c}_{,uv} = \frac{\partial^2 \mathbf{c}}{\partial u \partial v}.$$

Differentiating $(g_1, g_2)^T = \mathbf{0}$ with respect to any primal variable $\mathbf{y}$ gives

$$\begin{bmatrix} \mathbf{c}_{,u} \cdot \mathbf{c}_{,u} + \mathbf{r} \cdot \mathbf{c}_{,uu} & \mathbf{c}_{,u} \cdot \mathbf{c}_{,v} + \mathbf{r} \cdot \mathbf{c}_{,uv} \\ \mathbf{c}_{,v} \cdot \mathbf{c}_{,u} + \mathbf{r} \cdot \mathbf{c}_{,uv} & \mathbf{c}_{,v} \cdot \mathbf{c}_{,v} + \mathbf{r} \cdot \mathbf{c}_{,vv} \end{bmatrix} \begin{bmatrix} \frac{\partial u}{\partial \mathbf{y}} \\ \frac{\partial v}{\partial \mathbf{y}} \end{bmatrix} = \begin{bmatrix} \mathbf{c}_{,u} \cdot \frac{\partial \mathbf{x}_p}{\partial \mathbf{y}} - \frac{\partial \mathbf{c}}{\partial \mathbf{y}} \cdot \mathbf{c}_{,u} \\ \mathbf{c}_{,v} \cdot \frac{\partial \mathbf{x}_p}{\partial \mathbf{y}} - \frac{\partial \mathbf{c}}{\partial \mathbf{y}} \cdot \mathbf{c}_{,v} \end{bmatrix}.$$

For the particle variable $\mathbf{y} = \mathbf{x}_p$, one has $\partial \mathbf{c} / \partial \mathbf{y} = 0$ and $\partial \mathbf{x}_p / \partial \mathbf{y} = \mathbf{I}$, hence

$$\begin{bmatrix} \frac{\partial u}{\partial \mathbf{x}_p} \\ \frac{\partial v}{\partial \mathbf{x}_p} \end{bmatrix} = \mathbf{M}^{-1} \begin{bmatrix} \mathbf{c}_{,u} \\ \mathbf{c}_{,v} \end{bmatrix},$$

where

$$\mathbf{M} = \begin{bmatrix} \mathbf{c}_{,u} \cdot \mathbf{c}_{,u} + \mathbf{r} \cdot \mathbf{c}_{,uu} & \mathbf{c}_{,u} \cdot \mathbf{c}_{,v} + \mathbf{r} \cdot \mathbf{c}_{,uv} \\ \mathbf{c}_{,u} \cdot \mathbf{c}_{,v} + \mathbf{r} \cdot \mathbf{c}_{,uv} & \mathbf{c}_{,v} \cdot \mathbf{c}_{,v} + \mathbf{r} \cdot \mathbf{c}_{,vv} \end{bmatrix}.$$

This is exactly the code-level formula returned by `Dknot_div_Dpoint_Surface(pointer, tangent_u, tangent_v, curvature_uu, curvature_vv, curvature_uv)`.

For a control point $\mathbf{P}_a$, one has $\partial \mathbf{c} / \partial \mathbf{P}_a = N_a \mathbf{I}$ at fixed (u, v) , and the differentiated orthogonality equations produce the mixed terms involving

$$\frac{\partial N_a}{\partial u}, \quad \frac{\partial N_a}{\partial v}.$$

After elimination of $\partial u / \partial \mathbf{P}_a$ and $\partial v / \partial \mathbf{P}_a$, the code stores the result directly in `D2distance2_div_DctrlptDpoint` and `D2distance2_div_D2ctrlpt_Surface(...)`.

Therefore the second derivatives of the squared distance contain two parts:

1. explicit differentiation of $\mathbf{r} = \mathbf{c}(u, v) - \mathbf{x}_p$,
2. implicit differentiation of the closest-point parameters (u, v) through the matrix $\mathbf{M}^{-1}$ above.

This is why `PointNurbsDerivative` is the correct place where the implicit-differentiation contribution enters the barrier Hessian.

The first derivatives used in code are

$$\frac{\partial s}{\partial \mathbf{x}_p} = -2\mathbf{r}, \quad \frac{\partial s}{\partial \mathbf{P}_a} = 2N_a \mathbf{r}.$$

Hence

$$\frac{\partial \Psi_b}{\partial \mathbf{x}_p} = \frac{B'(d)}{2d} \frac{\partial s}{\partial \mathbf{x}_p}, \quad \frac{\partial \Psi_b}{\partial \mathbf{P}_a} = \frac{B'(d)}{2d} \frac{\partial s}{\partial \mathbf{P}_a}.$$

The full Hessian used in `assemble_barrier_matrix()` is

$$\frac{\partial^2 \Psi_b}{\partial \mathbf{q}_1 \partial \mathbf{q}_2} = \frac{1}{4} \left(\frac{B''(d)}{d^2} - \frac{B'(d)}{d^3} \right) \frac{\partial s}{\partial \mathbf{q}_1} \left(\frac{\partial s}{\partial \mathbf{q}_2} \right)^T + \frac{1}{2} \frac{B'(d)}{d} \frac{\partial^2 s}{\partial \mathbf{q}_1 \partial \mathbf{q}_2},$$

for any pair of variables $\mathbf{q}_1, \mathbf{q}_2 \in \{\mathbf{x}_p, \mathbf{P}_a, \mathbf{P}_b, \dots\}$.

In particular, the code assembles:

- control-point/control-point blocks

$$\mathbf{H}_{ab}^{pp} = \nabla_{\mathbf{P}_a \mathbf{P}_b}^2 \Psi_b,$$

- control-point/particle blocks

$$\mathbf{H}_{ap}^{pq} = \nabla_{\mathbf{P}_a \mathbf{x}_p}^2 \Psi_b,$$

- particle/particle block

$$\mathbf{H}_{pp}^{qq} = \nabla_{\mathbf{x}_p \mathbf{x}_p}^2 \Psi_b.$$

All geometric second derivatives are provided by `PointNurbsDerivative`, namely:

$$\frac{\partial^2 s}{\partial \mathbf{x}_p^2}, \quad \frac{\partial^2 s}{\partial \mathbf{P}_a \partial \mathbf{x}_p}, \quad \frac{\partial^2 s}{\partial \mathbf{P}_a \partial \mathbf{P}_b}.$$

These terms contain both the explicit dependence on the pointer $\mathbf{r}$ and the implicit dependence of the closest-point parameters (u, v) on the primal variables.

4.3 Friction energy for MPM-IGA contact

The friction initializer in `initialize_contact()` stores

$$\mu\lambda = \mu B'(d), \quad \mathbf{n} = \frac{\mathbf{r}}{\|\mathbf{r}\|},$$

up to the sign convention already absorbed in the code path.

The surface velocity is interpolated from control-point velocities:

$$\mathbf{v}_s(u, v) = \sum_a N_a(u, v) \dot{\mathbf{P}}_a.$$

The particle velocity is $\mathbf{v}_p$. The code forms the relative displacement increment

$$\Delta \mathbf{x} = (\mathbf{v}_p - \mathbf{v}_s) \Delta t.$$

Then the coupling friction energy in `calculate_friction_energy()` is

$$\Psi_f = \mu\lambda f_0(\|\mathbf{T}^T \Delta \mathbf{x}\|, \varepsilon_v, \Delta t), \quad \mathbf{T} = \mathbf{I} - \mathbf{n}\mathbf{n}^T.$$

4.4 Friction gradient and Hessian assembly

The code uses the velocity Jacobians

$$\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_p} = \mathbf{I}, \quad \frac{\partial \Delta \mathbf{x}}{\partial \mathbf{P}_a} = -N_a \mathbf{I},$$

through the helpers

$$\text{Dvelocity_div_Dpoint}() = \mathbf{I}, \quad \text{Dvelocity_div_Dctrlpt}(N_a) = -N_a \mathbf{I}.$$

Let

$$\mathbf{g}_{\Delta x}^{\text{code}} = -\mu\lambda f_1(\bar{d}) \frac{\mathbf{T}\bar{\mathbf{x}}}{\bar{d}}, \quad \bar{\mathbf{x}} = \mathbf{T}^T \Delta \mathbf{x}, \quad \bar{d} = \|\bar{\mathbf{x}}\|.$$

Then the code contracts this local vector with the Jacobians above:

$$\frac{\partial \Psi_f}{\partial \mathbf{x}_p} = \left(\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_p} \right)^T \mathbf{g}_{\Delta x}^{\text{code}}, \quad \frac{\partial \Psi_f}{\partial \mathbf{P}_a} = \left(\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{P}_a} \right)^T \mathbf{g}_{\Delta x}^{\text{code}}.$$

Likewise, with the code-level local Hessian

$$\mathbf{H}_{\Delta x}^{\text{code}} = \mathbf{T} \left[\mu\lambda \left(\frac{f_1(\bar{d})}{\bar{d}} \mathbf{I} + \frac{f_2(\bar{d})\bar{d} - f_1(\bar{d})}{\bar{d}^3} \bar{\mathbf{x}}\bar{\mathbf{x}}^T \right) \right] \mathbf{T}^T,$$

the assembled coupling blocks are

$$\begin{aligned} \mathbf{H}_{ab}^{pp} &= \left(\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{P}_a} \right)^T \mathbf{H}_{\Delta x}^{\text{code}} \left(\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{P}_b} \right), \\ \mathbf{H}_{ap}^{pq} &= \left(\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{P}_a} \right)^T \mathbf{H}_{\Delta x}^{\text{code}} \left(\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_p} \right), \\ \mathbf{H}_{pp}^{qq} &= \left(\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_p} \right)^T \mathbf{H}_{\Delta x}^{\text{code}} \left(\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_p} \right). \end{aligned}$$

This is precisely what `assemble_friction_matrix()` does when it calls:

- `self.friction.gradient(..., dvelocity_dvertice1, ...)`
- `self.friction.gradient(..., dvelocity_dpoint, ...)`
- `self.friction.hessian(..., dvelocity_dvertice1, dvelocity_dvertice2, ...)`
- `self.friction.hessian(..., dvelocity_dvertice1, dvelocity_dpoint, ...)`
- `self.friction.hessian(..., dvelocity_dpoint, dvelocity_dpoint, ...)`

5. Summary

The implemented contact model in GeoTaichi can be summarized as follows:

1. Barrier terms are differentiated through the squared-distance map, which yields a compact generic formula for both gradient and Hessian.
2. IPC-style MPM friction follows the frozen-normal strategy used by `phyverse`.
3. MPM-IGA barrier contact uses closest-point geometry on the NURBS surface together with first and second derivatives supplied by `PointNurbsDerivative`.
4. MPM-IGA friction uses a relative displacement increment built from particle velocity minus NURBS-interpolated surface velocity, and assembles all matrix blocks by contracting the local friction tensor with the corresponding kinematic Jacobians.

This document is intended to mirror the current code, not to replace it with a different analytical convention.